

Project Report: Image Filtering Engine

Course: Java Object-Oriented Programming (VITyarthi Project)

Student Name: Prashant Priydarshi

Reg. No. : 25BAI11147

Institution: VIT Bhopal University

1. Problem Statement & Motivation

The Problem: Graphic design software and image manipulation tools are traditionally resource-heavy, requiring massive memory overhead and complex graphical user interfaces. For users who simply need to apply standard mathematical visual filters to images, this overhead is inefficient.

Why it Matters: A lightweight, modular command-line tool allows developers and students to apply matrix convolutions natively. By building this without external dependencies, it ensures high-performance batch processing and demonstrates core Object-Oriented principles.

2. Technical Approach

- **Input Source:** I utilized standard uncompressed image files (e.g., JPG, PNG), processed natively through Java's built-in `javax.imageio` and `java.awt.image` libraries.
 - **Core Modules Selected:** To move beyond simple sequential scripting, I developed distinct filter modules:
 - Grayscale (Colour averaging)
 - Sepia (Weighted mathematical colour mapping)
 - Box Blur (3x3 Grid spatial convolution)
 - **Algorithm & Design:** I implemented the **Strategy Design Pattern**. While a procedural approach uses messy if/else blocks for different filters, the Strategy pattern utilizes a common `ImageTransformer` interface, allowing the application to dynamically swap filtering algorithms at runtime.
-

3. Key Decisions & Architecture

- **Data Modelling:** Used a custom ImageMatrix class populated with Pixel objects to ensure the algorithmic logic was separated from raw byte-level image processing.
 - **Interface Expansion:** Used the ImageTransformer interface to standardize the input and output (taking an ImageMatrix and returning a new one). This allows for infinite scalability if new filters are added later.
 - **Execution Strategy:** Since this is a CLI tool, I used java.util.Scanner to create an interactive terminal flow, prompting the user for input paths, filter selection, and output destinations dynamically.
-

4. Challenges Faced & Solutions

- **The "Out of Bounds" Error:** Encountered ArrayIndexOutOfBoundsException when the 3x3 Box Blur algorithm attempted to sample pixels at the very edges of the image.
 - *Solution:* Implemented strict spatial boundary checks (`currentY >= 0 && currentY < height`) to ensure the convolution matrix only sampled valid pixels.
 - **RGB Value Overflow Error:** Experienced visual artifacting when mathematical formulas (like Sepia) pushed RGB values above the maximum byte limit of 255.
 - *Solution:* Utilized hard caps within the Pixel constructor to automatically clamp any computed Colour values between 0 and 255.
 - **File Handling Complexity:** I initially tested basic file paths, but observed crashes when users typed incorrect filenames (FileNotFoundException).
 - *Solution:* Added robust exception handling (try-catch blocks) inside the ImageIOHandler to gracefully warn the user instead of terminating the program.
-

5. Results & Analysis

- **Total Java Modules Built: 8**
- **Algorithmic Filters Supported: 3**

Analysis: The Strategy Pattern model showed a significant improvement over standard procedural programming. The separated module approach ensures that adding a new filter (like Edge Detection) requires zero modifications to the core Main or ImageIOHandler classes.

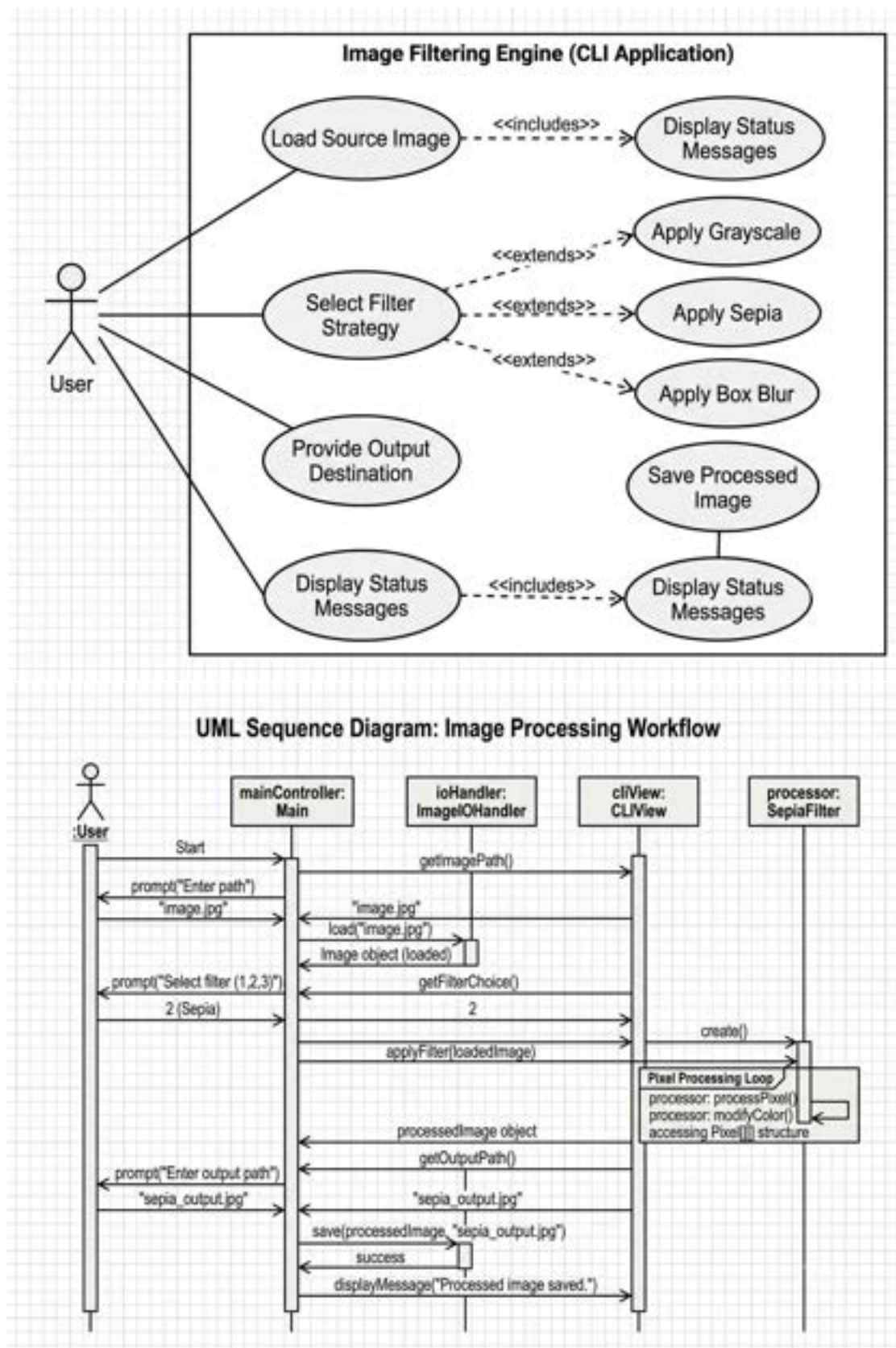


6. Conclusion & Learnings

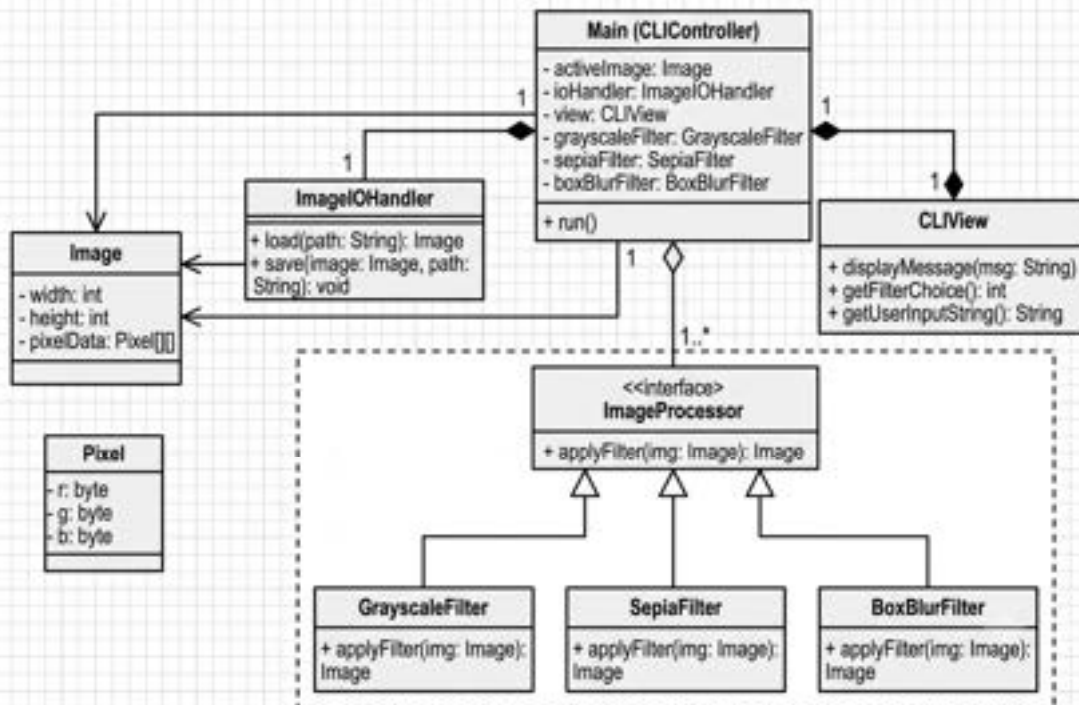
Through this project, I learned that:

- **Modularity is the engine:** Applying Object-Oriented interfaces (Strategy Pattern) is essential for modern, scalable software design.
- **Images are just math:** Visual effects like blurring and sepia are simply localized matrix convolutions and algebraic weighting.
- **Clean Code Matters:** Professional error handling (try-catch) and structured CLI prompts are just as important as the algorithm itself for making a tool usable by others.

7. UML Design Diagrams



UML Class Diagram: Image Filtering Engine Architecture



UML Activity Diagram: Command-Line Flow of Control (Main.java Logic)

